

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА ІМЕНІ
ЮРІЯ КОНДРАТЮКА»

Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 3

з навчальної дисципліни

"ТЕХНОЛОГІЇ РОЗРОБКИ КОРПОРАТИВНИХ WEB-ДОДАТКІВ"

Виконала:

ст.гр. 402-ТН

Чернушенко Валерія Юріївна

Перевірив:

Тютюнник Петро Богданович

Полтава 2026

Завдання для виконання:

Завдання

1. Реалізація шару доступу до даних (Data Access Layer)

- Налаштуйте підключення до бази даних через обрану ORM/ODM (Entity Framework, Hibernate, Mongoose, Drizzle тощо).
- Створіть моделі/сутності (Entities), що відповідають схемі БД з Лаб №2.
- Реалізуйте базові операції читання та запису (бажано через патерн **Repository** або ізольовані сервіси доступу до даних).

2. Реалізація бізнес-логіки (Service Layer)

- Створіть сервісний шар, який інкапсулює бізнес-правила.
- **Вимога:** API-контролери або резолвери не повинні містити складної логіки. Вони лише приймають дані, викликають сервіс і повертають результат клієнту.
- Застосуйте **Dependency Injection (DI)** для зв'язування компонентів.

3. Реалізація API (Оберіть один із варіантів) Виберіть підхід, який найкраще пасує до вашого тех стеку та вимог:

- **Варіант А: RESTful API**
 - Реалізуйте контролер для обраної сутності.
 - Забезпечте підтримку HTTP методів: GET (List/Single), POST (Create), PUT/PATCH (Update), DELETE.
 - Правильно використовуйте HTTP статус-коди (200, 201, 400, 404, 500).
 - Документація: **Swagger (OpenAPI)**.
- **Варіант Б: GraphQL API**
 - Опишіть схему (Schema) з типами даних.
 - Реалізуйте **Query** для отримання даних (аналог GET).
 - Реалізуйте **Mutation** для зміни даних (аналог POST/PUT/DELETE).
 - Налаштуйте **Resolvers** для підключення схеми до бізнес-логіки.
 - Інтерфейс: **GraphiQL** або **Apollo Sandbox**.
- **Варіант В: gRPC API**
 - Опишіть контракт сервісу у файлі .proto.

- Визначте повідомлення (Messages) для запитів та відповідей.
- Реалізуйте методи сервісу (Unary calls або Streaming, якщо необхідно).
- Інструментарій: підтримка генерації коду з proto-файлів.

4. Використання DTO (Data Transfer Objects)

- Незалежно від обраного протоколу, **заборонено** повертати клієнту "сирі" сутності бази даних.
- Використовуйте DTO (або message у gRPC, Type/Input у GraphQL) для передачі даних.
- Налаштуйте мапінг між сутностями БД та моделями API.

Очікуваний результат (Звіт та код):

1. Посилання на Git-репозиторій.

2. Скріншоти роботи API:

- Для REST: Swagger UI або Postman.
- Для GraphQL: Скріншот запиту (Query) та мутації (Mutation) у Playground.
- Для gRPC: Скріншот виклику методу через gRPC клієнт (напр. Postman gRPC або BloomRPC).

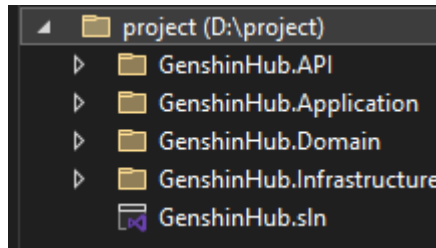
3. Приклади коду (фрагменти у звіті):

- Визначення моделі API (DTO / Proto message / GraphQL Type).
- Реалізація методу (Controller Action / Resolver / gRPC Service method).

Бонус (+2 бали): Реалізація валідації вхідних даних (наприклад, FluentValidation або Data Annotations), щоб API повертав чіткі повідомлення про помилки при некоректних запитах.

Структура проєкту

Проект організовано за принципами Clean Architecture:



Реалізація шару доступу до даних (Data Access Layer)

Сутності (Domain)

Приклад сутності **Character**:

```
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace GenshinHub.Domain.Entities;

public class Character
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string Id { get; set; } = string.Empty;

    public string Name { get; set; } = string.Empty;
    public string Element { get; set; } = string.Empty;
    public int Rarity { get; set; }
    public string WeaponType { get; set; } = string.Empty;
    public string? ImageUrl { get; set; }

    public List<CharacterMaterial> Materials { get; set; } = new();
}
```

Аналогічно створено сутності **User, Weapon, Material, News**.

MongoDB контекст

```
using Microsoft.Extensions.Configuration;
```

```
using MongoDB.Driver;
```

```
using GenshinHub.Domain.Entities;
```

```
namespace GenshinHub.Infrastructure.Persistence;
```

```
public class MongoDBContext
```

```
{
```

```
    private readonly IMongoDatabase _database;
```

```
    public MongoDBContext(IConfiguration configuration)
```

```
    {
```

```
        var connectionString = configuration.GetConnectionString("MongoDB");
```

```
        var client = new MongoClient(connectionString);
```

```
        _database = client.GetDatabase("GenshinHub");
```

```
    }
```

```
    public IMongoCollection<Character> Characters =>  
_database.GetCollection<Character>("characters");
```

```
    public IMongoCollection<User> Users =>  
_database.GetCollection<User>("users");
```

```
    public IMongoCollection<Weapon> Weapons =>  
_database.GetCollection<Weapon>("weapons");
```

```
    public IMongoCollection<Material> Materials =>  
_database.GetCollection<Material>("materials");
```

```
    public IMongoCollection<News> News =>  
_database.GetCollection<News>("news");
```

```
}
```

Репозиторії (патерн Repository)

Інтерфейс **ICharacterRepository**:

```
namespace GenshinHub.Domain.Interfaces;
```

```
public interface ICharacterRepository
{
    Task<IEnumerable<Character>> GetAllAsync();
    Task<Character?> GetByIdAsync(string id);
    Task CreateAsync(Character character);
    Task UpdateAsync(Character character);
    Task DeleteAsync(string id);
}
```

Реалізація:

```
using MongoDB.Driver;
using GenshinHub.Domain.Entities;
using GenshinHub.Domain.Interfaces;
using GenshinHub.Infrastructure.Persistence;

namespace GenshinHub.Infrastructure.Repositories;
```

```
public class CharacterRepository : ICharacterRepository
{
    private readonly IMongoCollection<Character> _characters;

    public CharacterRepository(MongoDbContext context)
    {
        _characters = context.Characters;
    }
}
```

```

public async Task<IEnumerable<Character>> GetAllAsync() =>
    await _characters.Find(_ => true).ToListAsync();

public async Task<Character?> GetByIdAsync(string id) =>
    await _characters.Find(c => c.Id == id).FirstOrDefaultAsync();

public async Task CreateAsync(Character character) =>
    await _characters.InsertOneAsync(character);

public async Task UpdateAsync(Character character) =>
    await _characters.ReplaceOneAsync(c => c.Id == character.Id, character);

public async Task DeleteAsync(string id) =>
    await _characters.DeleteOneAsync(c => c.Id == id);
}

```

Аналогічно створено репозиторії для інших сутностей (**UserRepository**, **WeaponRepository**, **MaterialRepository**, **NewsRepository**).

Реалізація бізнес-логіки (Service Layer)

DTO (Data Transfer Objects)

```

public class CharacterDto
{
    public string Id { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public string Element { get; set; } = string.Empty;
    public int Rarity { get; set; }
    public string WeaponType { get; set; } = string.Empty;
    public string? ImageUrl { get; set; }
    public List<CharacterMaterialDto> Materials { get; set; } = new();
}

```

```

public class CreateCharacterDto
{
    public string Name { get; set; } = string.Empty;
    public string Element { get; set; } = string.Empty;
    public int Rarity { get; set; }
    public string WeaponType { get; set; } = string.Empty;
    public string? ImageUrl { get; set; }
    public List<CharacterMaterialDto> Materials { get; set; } = new();
}

```

AutoMapper профіль

```

using AutoMapper;
using GenshinHub.Application.DTOs;
using GenshinHub.Domain.Entities;

namespace GenshinHub.Application.Mappings;

public class MappingProfile : Profile
{
    public MappingProfile()
    {
        CreateMap<Character, CharacterDto>().ReverseMap();
        CreateMap<CreateCharacterDto, Character>();
        CreateMap<CharacterMaterial, CharacterMaterialDto>().ReverseMap();

        CreateMap<User, UserDto>();
        CreateMap<UserProgress, UserProgressDto>();
        CreateMap<Talents, TalentsDto>();

        CreateMap<Weapon, WeaponDto>().ReverseMap();
    }
}

```



```

CreateMap<CreateWeaponDto, Weapon>();
CreateMap<WeaponMaterial, WeaponMaterialDto>().ReverseMap();

CreateMap<Material, MaterialDto>().ReverseMap();
CreateMap<CreateMaterialDto, Material>();

CreateMap<News, NewsDto>().ReverseMap();
CreateMap<CreateNewsDto, News>();
}
}

```

Cepbic (CharacterService)

```

using AutoMapper;
using GenshinHub.Application.DTOs;
using GenshinHub.Domain.Entities;
using GenshinHub.Domain.Interfaces;

namespace GenshinHub.Application.Services;

public class CharacterService
{
    private readonly ICharacterRepository _characterRepository;
    private readonly IMapper _mapper;

    public CharacterService(ICharacterRepository characterRepository, IMapper
mapper)
    {
        _characterRepository = characterRepository;
        _mapper = mapper;
    }
}

```

```
public async Task<IEnumerable<CharacterDto>> GetAllCharactersAsync()
{
    var characters = await _characterRepository.GetAllAsync();
    return _mapper.Map<IEnumerable<CharacterDto>>(characters);
}
```

```
public async Task<CharacterDto?> GetCharacterByIdAsync(string id)
{
    var character = await _characterRepository.GetByIdAsync(id);
    return character == null ? null : _mapper.Map<CharacterDto>(character);
}
```

```
public async Task<CharacterDto> CreateCharacterAsync(CreateCharacterDto
createDto)
{
    var character = _mapper.Map<Character>(createDto);
    await _characterRepository.CreateAsync(character);
    return _mapper.Map<CharacterDto>(character);
}
```

```
public async Task<bool> UpdateCharacterAsync(string id, CreateCharacterDto
updateDto)
{
    var existing = await _characterRepository.GetByIdAsync(id);
    if (existing == null) return false;

    _mapper.Map(updateDto, existing);
    await _characterRepository.UpdateAsync(existing);
    return true;
}
```

```

public async Task<bool> DeleteCharacterAsync(string id)
{
    var existing = await _characterRepository.GetByIdAsync(id);
    if (existing == null) return false;

    await _characterRepository.DeleteAsync(id);
    return true;
}
}

```

Аналогічні сервіси створено для **Weapon, Material, News, Auth, Progress, ResourceCalculation, User**.

Реалізація API (RESTful)

Контролер CharactersController

[ApiController]

[Route("api/[controller]")]

public class CharactersController : ControllerBase

```

{
    private readonly CharacterService _characterService;

    public CharactersController(CharacterService characterService)
    {
        _characterService = characterService;
    }
}

```

[HttpGet]

public async Task<IActionResult> GetAll()

```

{
    var characters = await _characterService.GetAllCharactersAsync();
    return Ok(characters);
}

```

```
}
```

```
[HttpGet("{id}")]
```

```
public async Task<IActionResult> GetById(string id)
```

```
{
```

```
    var character = await _characterService.GetCharacterByIdAsync(id);
```

```
    if (character == null) return NotFound();
```

```
    return Ok(character);
```

```
}
```

```
[HttpPost]
```

```
[Authorize(Roles = "admin")]
```

```
public async Task<IActionResult> Create([FromBody] CreateCharacterDto  
createDto)
```

```
{
```

```
    if (!ModelState.IsValid) return BadRequest(ModelState);
```

```
    var character = await _characterService.CreateCharacterAsync(createDto);
```

```
    return CreatedAtAction(nameof(GetById), new { id = character.Id }, character);
```

```
}
```

```
[HttpPut("{id}")]
```

```
[Authorize(Roles = "admin")]
```

```
public async Task<IActionResult> Update(string id, [FromBody]  
CreateCharacterDto updateDto)
```

```
{
```

```
    var updated = await _characterService.UpdateCharacterAsync(id, updateDto);
```

```
    if (!updated) return NotFound();
```

```
    return NoContent();
```

```
}
```

```

[HttpDelete("{id}")]
[Authorize(Roles = "admin")]
public async Task<IActionResult> Delete(string id)
{
    var deleted = await _characterService.DeleteCharacterAsync(id);
    if (!deleted) return NotFound();
    return NoContent();
}
}

```

Аналогічно реалізовано:

- **AuthController** (реєстрація, логін, JWT)
- **UserProgressController** (управління прогресом гравця)
- **AdminController** (керування користувачами)
- **WeaponsController, MaterialsController, NewsController.**

Налаштування **Dependency Injection** та **Swagger**

У **Program.cs** зареєстровано всі необхідні сервіси, репозиторії, AutoMapper, FluentValidation, JWT, Swagger:

```

using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using FluentValidation.AspNetCore;
using GenshinHub.Application.Mappings;
using GenshinHub.Application.Services;
using GenshinHub.Application.Validators;
using GenshinHub.Domain.Interfaces;
using GenshinHub.Infrastructure.Persistence;
using GenshinHub.Infrastructure.Repositories;

```

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers()
    .AddFluentValidation(fv =>
fv.RegisterValidatorsFromAssemblyContaining<CreateCharacterDtoValidator>());

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "Genshin Impact Player Hub
API", Version = "v1" });
    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        In = ParameterLocation.Header,
        Description = "Введіть JWT токен",
        Name = "Authorization",
        Type = SecuritySchemeType.Http,
        BearerFormat = "JWT",
        Scheme = "bearer"
    });
    c.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference { Type =
ReferenceType.SecurityScheme, Id = "Bearer" }
            },
            new string[] {}
        }
    }
}

```

```

    });
});

builder.Services.AddAutoMapper(typeof(MappingProfile));

builder.Services.AddSingleton<MongoDbContext>();

builder.Services.AddScoped<ICharacterRepository, CharacterRepository>();
builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddScoped<IWeaponRepository, WeaponRepository>();
builder.Services.AddScoped<IMaterialRepository, MaterialRepository>();
builder.Services.AddScoped<INewsRepository, NewsRepository>();

builder.Services.AddScoped<CharacterService>();
builder.Services.AddScoped<AuthService>();
builder.Services.AddScoped<ProgressService>();
builder.Services.AddScoped<ResourceCalculationService>();
builder.Services.AddScoped<UserService>();
builder.Services.AddScoped<WeaponService>();
builder.Services.AddScoped<MaterialService>();
builder.Services.AddScoped<NewsService>();

var jwtKey = builder.Configuration["Jwt:Key"];
var key = Encoding.UTF8.GetBytes(jwtKey!);
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,

```

```

        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = builder.Configuration["Jwt:Issuer"],
        ValidAudience = builder.Configuration["Jwt:Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(key)
    };
});

```

```
builder.Services.AddAuthorization();
```

```
var app = builder.Build();
```

```

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

```

```

app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

```

```
app.Run();
```

Валідація вхідних даних

Для кожного DTO створено валідатор. Приклад для **CreateCharacterDtoValidator**:

```

public class CreateCharacterDtoValidator : AbstractValidator<CreateCharacterDto>
{

```



```

public CreateCharacterDtoValidator()
{
    RuleFor(x => x.Name)
        .NotEmpty().WithMessage("Ім'я персонажа обов'язкове")
        .MaxLength(100);

    RuleFor(x => x.Element)
        .NotEmpty()
        .Must(e => new[] { "Pyro", "Hydro", "Anemo", "Electro", "Dendro", "Cryo",
"Geo" }.Contains(e))
        .WithMessage("Невірний елемент");

    RuleFor(x => x.Rarity)
        .InclusiveBetween(4, 5)
        .WithMessage("Рідкість має бути 4 або 5");

    RuleFor(x => x.WeaponType)
        .NotEmpty();
}
}

```

Валідатори реєструються в **Program.cs** через **AddFluentValidation**.

Тестування API

Запуск



```

Developer PowerShell
+ Developer PowerShell - [?] [?] [?]
PS D:\project> cd GenshinHub.API
PS D:\project\GenshinHub.API> dotnet run
Using launch settings from D:\project\GenshinHub.API\Properties\launchSettings.json...
Building...
D:\project\GenshinHub.Application\Services\AuthService.cs(67,67): warning CS8604: Possible null reference argument for parameter 's' in 'byte[] Encoding.GetBytes(string s)'.
D:\project\GenshinHub.API\Program.cs(16,32): warning CS8618: 'FluentValidationVcConfiguration.RegisterValidatorsFromAssemblyContaining(T)(Func<AssemblyScanner.AssemblyScanResult, bool>, ServiceLifetime, bool)' is obsolete: 'RegisterValidatorsFromAssemblyContaining is deprecated. Call services.AddValidatorsFromAssemblyContaining(T) instead, which has the same effect. See https://github.com/FluentValidation/FluentValidation/issues/1965'
D:\project\GenshinHub.API\Program.cs(15,1): warning CS8618: 'FluentValidationVcExtensions.AddFluentValidation(DhcBuilder, Action<FluentValidationVcConfiguration>)' is obsolete: 'Calling AddFluentValidation() is deprecated. Call services.AddFluentValidationAutoValidation().AddFluentValidationClientsideAdapters() instead, which has the same effect. For details see https://github.com/FluentValidation/FluentValidation/issues/1965'
info: Microsoft.Hosting.Lifetime[14]
      Now listening on http://localhost:5016
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\project\GenshinHub.API

```

Swagger

Swagger UI interface for the Genshin Impact Player Hub API. The browser address bar shows `localhost:5016/swagger/index.html`. The Swagger logo and "Select a definition" dropdown (showing "GenshinHub.API.v1") are at the top.

The API title is **Genshin Impact Player Hub API** with a **GA33** badge. Below it is the URL `http://localhost:5016/swagger/index.html` and an **Authorize** button.

The API is organized into several sections, each with a list of endpoints:

- Admin**
 - GET `/api/Admin/users`
 - GET `/api/Admin/users/{id}`
 - PUT `/api/Admin/users/{id}/role`
- Auth**
 - POST `/api/Auth/register`
 - POST `/api/Auth/login`
- Characters**
 - GET `/api/Characters`
 - POST `/api/Characters`
 - GET `/api/Characters/{id}`
 - PUT `/api/Characters/{id}`
 - DELETE `/api/Characters/{id}`
- Materials**
 - GET `/api/Materials`
 - POST `/api/Materials`

The following sections are expanded to show their endpoints:

- Materials**
 - GET `/api/Materials`
 - POST `/api/Materials`
 - GET `/api/Materials/{id}`
 - PUT `/api/Materials/{id}`
 - DELETE `/api/Materials/{id}`
- News**
 - GET `/api/News`
 - POST `/api/News`
 - GET `/api/News/recent/{count}`
 - GET `/api/News/{id}`
 - PUT `/api/News/{id}`
 - DELETE `/api/News/{id}`
- UserProgress**
 - GET `/api/UserProgress/{characterId}`
 - PUT `/api/UserProgress/{characterId}`
 - POST `/api/UserProgress/{characterId}`
 - GET `/api/UserProgress/resources/calculate`
- Weapons**
 - GET `/api/Weapons`
 - POST `/api/Weapons`
 - GET `/api/Weapons/{id}`
 - PUT `/api/Weapons/{id}`
 - DELETE `/api/Weapons/{id}`

Схеми DTOs

Schemas	^
CharacterMaterialDto	>
CreateCharacterDto	>
CreateMaterialDto	>
CreateNewsDto	>
CreateWeaponDto	>
LoginDto	>
RegisterDto	>
TalentsDto	>
UserProgressDto	>
WeaponMaterialDto	>

POST/api/Auth/register

[illegible]

[MyCluster](#) > [GenshinHub](#) > [users](#)

Documents 1 Aggregations Schema Indexes 1 Validation

🕒 ▼ Type a query: { field: 'value' } or [Generate query](#) 🚀

[+ ADD DATA](#)
[UPDATE](#)
[DELETE](#)
[EXPORT CODE](#)

```

    _id: ObjectId('69cc0173bb2e53093acccecb')
    Email: "someEmail@example.com"
    PasswordHash: "$2a$11$iA1kGwk6pQu6ZUSmIjuMsO3XxG/6vqWfNV7AL.w2xJvJFz6N3N0u0"
    Role: "player"
    CreatedAt: 2026-03-31T17:16:35.279+00:00
    Progress: Array (empty)

```

[illegible]

Available authorizations

×

Bearer (http, Bearer)

Введіть JWT токен

Value:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0=

Authorize

Close

Available authorizations

×

Bearer (http, Bearer)

Authorized

Введіть JWT токен

Value: *****

Logout

Close

POST/api/Characters

Responses

Curl

```
curl -X 'POST' \
'http://localhost:5016/api/Characters' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodm8wOjEvc2NoZmlkcyS4bGxkb2FwLnB9Zy93cy8yMDAxMTAzA121kzW5oaXRSL2NsYWItcy9uYVllamNlbnRpZmllciI6IjVSZWY2MnAtczYmIyZTUzMDkzYWVhcnJvY2VjYyIiImh0dHAg6Ly9' \
-H 'Content-Type: application/json' \
-d '{
  "name": "Diluc",
  "element": "Pyro",
  "rarity": 5,
  "weaponType": "Claymore",
  "imageUrl": "https://example.com/diluc.png",
  "materials": [
    {
      "materialId": "32cbfe0fb2e53893adcfc6",
      "name": "Agnidus Agate Silver",
      "category": "boss",
      "quantityPerLevel": 1
    }
  ]
}
```

Request URL

http://localhost:5016/api/Characters

Server response

Code	Details
------	---------

403

400
Undocumented

403
Undocumented Error: Forbidden

Response headers

```
content-length: 0
date: Tue, 31 Mar 2026 17:18:38 GMT
server: Kestrel
```

Дали адміна

```

_id: ObjectId('69cc0173bb2e53093accceb')
Email: "someEmail@example.com"
PasswordHash: "$2a$11$iA1kGwk6pQu6ZUSmIjuMs03XxG/6vqWFnV7AL.w2xJvJFz6N3N0u0"
Role: "admin"
CreatedAt: 2026-03-31T17:16:35.279+00:00
Progress: Array (empty)

```

POST/api/Auth/login

Responses

Curl

[illegible]

Request URL

http://localhost:5016/api/Auth/login

Server response

Code	Details
------	---------

200

Response body

[illegible]

Discussion

Available authorizations



Bearer (http, Bearer)

Authorized

Введіть JWT токен

Value: *****

Logout

Close

Available authorizations



Bearer (http, Bearer)

Введіть JWT токен

Value:

4FLInhfvuguoXbV5gxchTey-s

Authorize

Close

Available authorizations



Bearer (http, Bearer)

Authorized

Введіть JWT токен

Value: *****

Logout

Close

[illegible]

Documents1

Aggregations

Schema

Index

Type a query: { field: 'value' } or [Gen](#)

+ ADD DATA

UPDATE

DELETE

EXPORT

```
_id: ObjectId('69cc0283bb2e53093acccecc')
Name: "Diluc"
Element: "Pyro"
Rarity: 5
WeaponType: "Claymore"
ImageUrl: "https://example.com/diluc.png"
▼ Materials: Array (1)
  ▼ 0: Object
    MaterialId: "32cbfe0fbb2e53093adcfkc6"
    Name: "Agnidus Agate Sliver"
    Category: "boss"
    QuantityPerLevel: 1
```

Валідація

POST/api/Characters

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5016/api/Characters' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLn9yZy93cy8yMDAxLzA1L2lkZS0aXRSI2NsYWltcy9uYW1laHRlbnRpZm11ciI6IjY5Y2NmMTczYmIyZTUzMDkzYW9jY2VjYiIsInh0dHA6Ly93aW9kaWVzIjoiIn0=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLn9yZy93cy8yMDAxLzA1L2lkZS0aXRSI2NsYWltcy9uYW1laHRlbnRpZm11ciI6IjY5Y2NmMTczYmIyZTUzMDkzYW9jY2VjYiIsInh0dHA6Ly93aW9kaWVzIjoiIn0=' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Diluc",
    "element": "123",
    "rarity": 0,
    "weaponType": "Claymore",
    "imageId": "https://example.com/diluc.png",
    "materials": [
      {
        "materialId": "32cbfe0fbb2e53093adcfc6",
        "name": "Agnidus Agate Sliver",
        "category": "Boss",
        "quantityPerLevel": 1
      }
    ]
  }'
```

Request URL

http://localhost:5016/api/Characters

Server response

Code

Details

400

Undocumented

Error: Bad Request

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Rarity": [
      "Рідкість має бути 4 або 5"
    ],
    "Element": [
      "Невірний елемент"
    ]
  },
  "traceId": "00-275e8e9ec496cff84dfe505679099eff-d70c67a6f9071472-00"
}
```

Download

POST/api/Auth/register

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5016/api/Auth/register' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLn9yZy93cy8yMDAxLzA1L2lkZS0aXRSI2NsYWltcy9uYW1laHRlbnRpZm11ciI6IjY5Y2NmMTczYmIyZTUzMDkzYW9jY2VjYiIsInh0dHA6Ly93aW9kaWVzIjoiIn0=' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "someEmailcom",
    "password": "1"
  }'
```

Request URL

http://localhost:5016/api/Auth/register

Server response

Code

Details

400

Undocumented

Error: Bad Request

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Email": [
      "Невірний формат email"
    ],
    "Password": [
      "Пароль має містити мінімум 6 символів"
    ]
  },
  "traceId": "00-2e5ebb9671f2a0b983032c27e2e64b1d-2278f7864123d6b4-00"
}
```

Download

GET/api/Characters

Curl

```
curl -X 'GET' \
  'http://localhost:5016/api/Characters' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWZlbnR1bWVjY3cy8yMDAxLzA1L2lkZS0aXR5L2NsYWltcy9uYm11aHRlbnRpbmZm11ciI6IjY5Y2%MTczYmIyZTUzMDkzYmNjY2VjYiIsImh0dHA6Ly9...
```

Request URL

http://localhost:5016/api/Characters

Server response

Code	Details
200	Response body <pre>{ { "id": "69cc0283bb2e53093acccecc", "name": "Diluc", "element": "Pyro", "rarity": 5, "weaponType": "Claymore", "imageUrl": "https://example.com/diluc.png", "materials": [{ "materialId": "32cbfe0fbb2e53093adcfkc6", "name": "Agnidus Agate Sliver", "category": "boss", "quantityPerLevel": 1 }] } }</pre>

Download

PUT/api/Characters/{id}

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:5016/api/Characters/69cc0283bb2e53093acccecc' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWZlbnR1bWVjY3cy8yMDAxLzA1L2lkZS0aXR5L2NsYWltcy9uYm11aHRlbnRpbmZm11ciI6IjY5Y2%MTczYmIyZTUzMDkzYmNjY2VjYiIsImh0dHA6Ly9...' \
  -H 'Content-type: application/json' \
  -d '{
    "name": "Not Diluc",
    "element": "Cryo",
    "rarity": 4,
    "weaponType": "Greatsword",
    "imageUrl": "https://example.com/notdiluc.png",
    "materials": [
      {
        "materialId": "32cbfe0fbb2e53093adcfkc6",
        "name": "Agnidus Agate Sliver",
        "category": "boss",
        "quantityPerLevel": 3
      }
    ]
  }'
```

Request URL

http://localhost:5016/api/Characters/69cc0283bb2e53093acccecc

Server response

Code	Details
204	Response headers <pre>date: Tue, 31 Mar 2026 17:25:10 GMT server: Kestrel</pre>

Download

MyCluster > GenshinHub > characters

Documents 1

Aggregations

Schema

Index

Type a query: { field: 'value' } or [Generate](#)

ADD DATA

UPDATE

DELETE

EXPORT

```
_id: ObjectId('69cc0283bb2e53093acccecc')
Name : "Not Diluc"
Element : "Cryo"
Rarity : 4
WeaponType : "Greatsword"
ImageUrl : "https://example.com/notdiluc.png"
Materials : Array (1)
  0: Object
    MaterialId : "32cbfe0fbb2e53093adcfkc6"
    Name : "Agnidus Agate Sliver"
    Category : "boss"
    QuantityPerLevel : 3
```

DELETE/api/Characters/{id}

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:5016/api/Characters/69cc0283bb2e53093accecc' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWozb2FvLn9yZy93cy8yMDAxLzA1L2lkZm50aXR5L2NsYWltcy9uYm11ahR1bnRpZm11ciI6IjY5Y2%MTczYmIyZTUzMDkzYmNjY2VjYiIsImh0dHkiOiJyY9'

```

Request URL

http://localhost:5016/api/Characters/69cc0283bb2e53093accecc

Server response

Code	Details
204	Response headers date: Tue, 31 Mar 2026 17:26:38 GMTserver: Kestrel

MyCluster > GenshinHub > characters

Documents 0

Aggregations

Schema

Indexes 1

Validation

🕒

Type a query: { field: 'value' } or [Generate query](#) ⚙️

➕ ADD DATA ▾

✎ UPDATE

🗑 DELETE

📄 EXPORT CODE



This collection has no data

It only takes a few seconds to import data from a JSON or CSV file.

GET/api/Characters

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5016/api/Characters' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWozb2FvLn9yZy93cy8yMDAxLzA1L2lkZm50aXR5L2NsYWltcy9uYm11ahR1bnRpZm11ciI6IjY5Y2%MTczYmIyZTUzMDkzYmNjY2VjYiIsImh0dHkiOiJyY9'

```

Request URL

http://localhost:5016/api/Characters

Server response

Code	Details
200	Response body []  Download

Відповіді на контрольні запитання

1. Порівняйте REST, GraphQL та gRPC. У яких випадках доцільно використовувати кожен із них?

REST — це архітектурний стиль, що базується на HTTP-методах та ресурсах. Він найкраще підходить для простих CRUD-операцій, коли структура даних стабільна і клієнти потребують стандартизованого підходу.

GraphQL — це мова запитів, яка дозволяє клієнту точно описати потрібні поля та зв'язки в одному запиті. Його доцільно використовувати при складних ієрархіях даних, коли різні клієнти мають різні потреби в полях, а також для уникнення надлишковості та множинних запитів.

gRPC — це високопродуктивний фреймворк, що використовує Protocol Buffers та HTTP/2. Він ідеально підходить для мікросервісної архітектури, внутрішніх комунікацій між сервісами, real-time додатків та сценаріїв, де критичними є низька затримка та компактність передачі даних.

2. Що таке проблема Over-fetching та Under-fetching даних? Як GraphQL вирішує ці проблеми порівняно з REST?

Проблема **Over-fetching** полягає в отриманні більшої кількості даних, ніж потрібно клієнту: наприклад, REST-ендпоїнт повертає всі поля об'єкта, навіть якщо клієнту потрібні лише деякі з них.

Проблема **Under-fetching** виникає, коли для отримання пов'язаних даних необхідно виконати кілька послідовних запитів (спочатку отримати список ресурсів, а потім кожен окремо).

GraphQL вирішує обидві проблеми, дозволяючи клієнту в одному запиті точно описати, які поля та зв'язки йому потрібні. Сервер повертає лише запитані дані, що виключає надлишковість і не вимагає додаткових запитів.

3. Що таке Protocol Buffers (Protobuf)? Які його переваги перед JSON?

Protocol Buffers (Protobuf) — це мовно-нейтральний, платформно-нейтральний формат серіалізації структурованих даних, розроблений Google. Він використовує компактне бінарне представлення, що забезпечує менший розмір даних у порівнянні з JSON. Крім того, Protobuf працює швидше завдяки бінарному кодуванню та підтримує строгу типізацію, що дозволяє автоматично генерувати код для різних мов програмування. Це спрощує розробку, зменшує ризик помилок і забезпечує зворотну сумісність при еволюції схеми.

4. Як забезпечується версіонування API у REST та GraphQL?

У **REST** версіонування найчастіше реалізують через URL (наприклад, `/api/v1/users`), через заголовки Асепт (наприклад, `application/vnd.myapi.v1+json`) або шляхом додавання нових полів без видалення старих. У **GraphQL** основна стратегія — це еволюція схеми: додаються нові поля, типи або аргументи, а старі позначаються директивою `@deprecated`, що дозволяє клієнтам поступово переходити на нові версії. Також можна тримати кілька версій схеми на різних кінцевих точках (наприклад, `/graphql/v1` та `/graphql/v2`), хоча такий підхід менш поширений.

5. Поясніть концепцію Idempotency (ідемпотентності). Які методи/операції мають бути ідемпотентними?

Ідемпотентність (Idempotency) — це властивість операції, при якій повторний виклик з тими ж параметрами дає той самий результат і не змінює стан системи після першого виконання.

У HTTP ідемпотентними мають бути методи **GET**, **PUT** та **DELETE**: повторний **GET** повертає ті ж дані, повторний **DELETE** не змінює стан після видалення. Метод **POST** зазвичай не є ідемпотентним, оскільки повторний запит може створити кілька ресурсів. Метод **PATCH** може бути ідемпотентним, якщо його реалізовано з урахуванням цієї властивості.

6. Яка роль Schema Definition Language (SDL) у GraphQL або .proto файлів у gRPC?

У GraphQL SDL (Schema Definition Language) використовується для опису типів даних, запитів (Query), мутацій (Mutation) та підписок (Subscription). Вона слугує контрактом між клієнтом і сервером, дозволяючи автоматичну генерацію документації, валідацію запитів та роботу з інструментами на кшталт GraphQL або Apollo Studio.

У gRPC аналогічну роль виконують .proto-файли, які визначають структуру повідомлень (messages) та сервісів (services). З них генерується код для різних мов програмування, що забезпечує строгу типізацію, швидку серіалізацію та легке підтримання змін контракту.